



Rapport de projet

Réalisé par

Haitham El Abdioui,

Application de Classification Intelligente de
Fruits et Légumes **FruitVision AI**

sous la direction de :

Mme Ibtissame AOURAGHE Encadrant Académique

Remerciements

Je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué à la réalisation de ce projet de fin d'études.

Tout d'abord, je souhaite remercier chaleureusement Madame **Ibtissame Aouraghe**, mon encadrante académique, pour son encadrement rigoureux, ses conseils avisés et sa disponibilité tout au long de ce projet. Ses orientations m'ont permis d'approfondir mes connaissances en intelligence artificielle et développement mobile.

Je remercie également l'ensemble du corps professoral pour la qualité de la formation dispensée qui m'a permis d'acquérir les compétences nécessaires à la réalisation de ce travail.

Mes remerciements vont aussi aux membres du jury qui ont accepté d'évaluer ce travail et dont les remarques constructives contribueront sans aucun doute à son amélioration.

Enfin, je tiens à remercier ma famille et mes amis pour leur soutien constant et leurs encouragements qui m'ont permis de mener à bien ce projet.

Résumé

L'intelligence artificielle et l'apprentissage automatique ont révolutionné de nombreux domaines, notamment la vision par ordinateur et la classification d'images. Ce projet présente le développement de **FruitVision AI**, une application mobile multiplateforme de classification de fruits et légumes utilisant des réseaux de neurones profonds.

L'objectif principal de ce projet est de concevoir et implémenter une solution complète intégrant des modèles d'apprentissage automatique performants (CNN et ANN) dans une application Flutter moderne et intuitive. L'application permet aux utilisateurs de prendre ou télécharger une photo d'un fruit ou légume et d'obtenir instantanément sa classification parmi cinq catégories : pomme, banane, carotte, orange et tomate.

Le projet adopte une architecture microservices avec une séparation claire entre le frontend mobile développé en Flutter et le backend de prédiction implémenté en Python avec FastAPI. Cette approche garantit une scalabilité optimale et facilite la maintenance du système. Pour la partie apprentissage automatique, nous avons exploité le transfer learning avec MobileNetV2, atteignant une précision de 93,62% sur notre ensemble de validation.

L'authentification des utilisateurs est gérée via Firebase Authentication, offrant une expérience sécurisée et fluide. L'interface utilisateur a été soigneusement conçue selon les principes du Material Design pour garantir une expérience utilisateur optimale sur toutes les plateformes.

Les résultats obtenus démontrent l'efficacité du transfer learning pour les tâches de classification d'images avec des datasets de taille limitée, et valident l'approche architecturale choisie pour le déploiement de modèles d'IA dans des applications mobiles.

Mots-clés : Intelligence Artificielle, Deep Learning, Classification d'Images, CNN, Transfer Learning, Flutter, Mobile Development, Firebase, Microservices, MobileNetV2

Introduction Générale

Contexte et Motivation

À l'ère du numérique, l'intelligence artificielle (IA) est devenue omniprésente dans notre quotidien. Des assistants virtuels aux systèmes de recommandation, en passant par les véhicules autonomes, l'IA transforme progressivement tous les secteurs d'activité. Parmi les domaines les plus prometteurs de l'IA, la vision par ordinateur connaît un développement particulièrement remarquable grâce aux avancées récentes en apprentissage profond (Deep Learning).

La classification d'images, l'une des tâches fondamentales de la vision par ordinateur, permet aux machines de reconnaître et catégoriser automatiquement le contenu visuel. Cette technologie trouve des applications dans de nombreux domaines : reconnaissance faciale, diagnostic médical assisté par ordinateur, contrôle qualité industriel, agriculture de précision, et bien d'autres.

Dans le domaine alimentaire et nutritionnel, la capacité à identifier automatiquement des fruits et légumes présente un intérêt particulier. Cette technologie peut servir de base à des applications d'aide nutritionnelle, de gestion de stocks en grande distribution, ou encore d'éducation alimentaire pour les enfants. Cependant, le déploiement de telles solutions sur des dispositifs mobiles pose des défis techniques importants en termes de performance, de consommation de ressources, et d'expérience utilisateur.

Problématique

Le développement d'une application mobile de classification d'images intelligente soulève plusieurs questions techniques et méthodologiques :

- Comment concevoir et entraîner des modèles de deep learning efficaces avec des ressources computationnelles et des datasets limités ?
- Comment intégrer ces modèles dans une architecture applicative mobile performante et scalable ?
- Quelle approche architecturale adopter pour assurer la séparation des préoccupations entre l'interface utilisateur et la logique de prédiction ?
- Comment garantir une expérience utilisateur fluide tout en maintenant une précision de classification acceptable ?
- Comment gérer l'authentification et la sécurité des données utilisateurs dans un contexte mobile ?

Objectifs du Projet

L'objectif principal de ce projet est de développer **FruitVision AI**, une application mobile complète de classification de fruits et légumes démontrant l'intégration réussie de techniques avancées d'apprentissage automatique dans un environnement mobile moderne.

Les objectifs spécifiques sont les suivants :

1. Développement de modèles d'apprentissage automatique performants :

- Concevoir et entraîner un réseau de neurones convolutif (CNN) pour la classification d'images
- Implémenter un réseau de neurones artificiel (ANN) pour comparaison
- Explorer le transfer learning avec des modèles pré-entraînés (MobileNetV2)
- Optimiser les performances pour atteindre une précision supérieure à 90%

2. Architecture et développement backend :

- Concevoir une API REST avec FastAPI pour servir les modèles
- Implémenter des endpoints de prédiction pour les deux modèles
- Gérer le prétraitement des images côté serveur
- Assurer la scalabilité et la performance du service

3. Développement de l'application mobile :

- Créer une interface utilisateur moderne et intuitive avec Flutter
- Implémenter la capture et le téléchargement d'images
- Intégrer les appels API pour la classification
- Gérer l'authentification des utilisateurs via Firebase
- Assurer la compatibilité multiplateforme (Android, iOS, Web)

4. Intégration et tests :

- Intégrer tous les composants dans un système cohérent
- Valider les performances end-to-end
- Tester l'expérience utilisateur sur différentes plateformes

Contributions

Ce projet apporte les contributions suivantes :

- Une démonstration pratique de l'efficacité du transfer learning pour la classification d'images avec des datasets de taille limitée
- Une architecture microservices complète pour le déploiement de modèles d'IA dans des applications mobiles
- Une application Flutter moderne intégrant Firebase et des APIs de machine learning
- Une étude comparative entre différentes architectures de réseaux de neurones (CNN vs ANN) pour la classification d'images
- Une documentation technique complète du processus de développement

Organisation du Rapport

Ce rapport est organisé en plusieurs chapitres :

- Le **Chapitre 2** présente l'état de l'art des technologies utilisées, incluant le deep learning, les réseaux de neurones convolutifs, le transfer learning, ainsi que les frameworks Flutter et Firebase.
- Le **Chapitre 3** détaille l'architecture du système et la stack technologique choisie, en expliquant les choix d'implémentation et la conception de chaque composant.
- Le **Chapitre 4** présente la mise en œuvre du projet, de la préparation des données à l'entraînement des modèles, en passant par le développement de l'API et de l'application mobile.
- Le **Chapitre 5** expose les résultats obtenus, incluant les performances des modèles, des captures d'écran de l'application, et une analyse des résultats.
- Le **Chapitre 6** propose une discussion critique des résultats, des limitations du projet et des perspectives d'amélioration.
- Enfin, la **Conclusion** résume les réalisations du projet et ouvre sur des perspectives futures.

Chapitre 1

État de l'Art

1.1 Intelligence Artificielle et Apprentissage Automatique

1.1.1 Fondements Théoriques

L'intelligence artificielle (IA) est un domaine de l'informatique visant à créer des systèmes capables d'effectuer des tâches nécessitant normalement l'intelligence humaine. L'apprentissage automatique (Machine Learning) constitue une branche de l'IA permettant aux ordinateurs d'apprendre à partir de données sans être explicitement programmés pour chaque tâche.

L'apprentissage automatique se divise en trois catégories principales :

- **Apprentissage supervisé** : Le modèle apprend à partir de données étiquetées, comme dans notre cas où chaque image de fruit/légume est associée à sa classe.
- **Apprentissage non supervisé** : Le modèle découvre des structures cachées dans des données non étiquetées.
- **Apprentissage par renforcement** : Le modèle apprend par essais et erreurs en interagissant avec un environnement.

Notre projet s'inscrit dans le cadre de l'apprentissage supervisé, où nous disposons d'un dataset d'images étiquetées pour entraîner nos modèles de classification.

1.1.2 Deep Learning et Réseaux de Neurones

Le deep learning, ou apprentissage profond, est une sous-catégorie de l'apprentissage automatique utilisant des réseaux de neurones artificiels à plusieurs couches. Cette approche s'inspire du fonctionnement du cerveau humain et a révolutionné de nombreux domaines, notamment la vision par ordinateur.

Un neurone artificiel effectue une somme pondérée de ses entrées, applique une fonction d'activation, et produit une sortie. Les réseaux de neurones empilent plusieurs couches de neurones, permettant d'extraire des représentations de plus en plus abstraites des données.

1.2 Réseaux de Neurones Convolutifs (CNN)

1.2.1 Architecture et Principes

Les Convolutional Neural Networks (CNN) sont des architectures spécialisées pour le traitement d'images. Inventés dans les années 1980 et popularisés par Yann LeCun avec LeNet-5, les CNN ont révolutionné la vision par ordinateur après la victoire d'AlexNet au concours ImageNet en 2012.

Un CNN typique se compose de plusieurs types de couches :

1. **Couches de convolution** : Appliquent des filtres (kernels) sur l'image pour extraire des features locales. Ces filtres détectent des motifs comme les bords, les textures, ou des formes plus complexes dans les couches profondes.
2. **Couches de pooling** : Réduisent la dimension spatiale des features maps, diminuant ainsi le nombre de paramètres et la charge computationnelle. Le max pooling et l'average pooling sont les méthodes les plus courantes.
3. **Couches de normalisation** : Comme le Batch Normalization, qui stabilise et accélère l'entraînement en normalisant les activations.
4. **Couches fully connected** : En fin de réseau, elles combinent toutes les features extraites pour effectuer la classification finale.
5. **Couches de dropout** : Technique de régularisation qui désactive aléatoirement des neurones pendant l'entraînement pour éviter le surapprentissage.

1.2.2 Avantages des CNN pour la Vision par Ordinateur

Les CNN présentent plusieurs avantages pour la classification d'images :

- **Invariance par translation** : Grâce aux convolutions, un CNN peut détecter un motif quelle que soit sa position dans l'image.
- **Partage des poids** : Les mêmes filtres sont appliqués sur toute l'image, réduisant considérablement le nombre de paramètres.
- **Hierarchie de features** : Les couches successives apprennent des représentations de plus en plus abstraites, des edges simples aux objets complexes.
- **Performance** : Les CNN obtiennent des résultats state-of-the-art sur la plupart des benchmarks de vision par ordinateur.

1.3 Transfer Learning

1.3.1 Principe et Motivation

Le transfer learning est une technique consistant à réutiliser un modèle pré-entraîné sur une tâche source (généralement ImageNet avec ses 1000 classes et millions d'images) pour l'adapter à une tâche cible différente avec moins de données.

Cette approche repose sur l'hypothèse que les features de bas niveau apprises sur ImageNet (détection de contours, textures, couleurs) sont génériques et transférables à d'autres tâches de vision par ordinateur.

1.3.2 Stratégies de Transfer Learning

Il existe plusieurs stratégies de transfer learning :

1. **Feature Extraction** : On gèle toutes les couches du modèle pré-entraîné et on remplace uniquement la couche de classification finale. Le modèle sert d'extracteur de features fixes.
2. **Fine-tuning** : On dégèle les dernières couches du modèle pré-entraîné et on les réentraîne conjointement avec la nouvelle couche de classification sur le nouveau dataset.
3. **Fine-tuning progressif** : On commence par la feature extraction, puis on dégèle progressivement les couches en partant de la fin pour affiner leur apprentissage.

Dans notre projet, nous avons adopté une approche combinant feature extraction initiale suivie de fine-tuning pour optimiser les performances.

1.3.3 MobileNetV2

MobileNetV2 est une architecture CNN optimisée développée par Google spécifiquement pour les applications mobiles et embarquées. Elle utilise des **depthwise separable convolutions** et des **inverted residual blocks** pour réduire drastiquement le nombre de paramètres et les calculs, tout en maintenant une précision compétitive.

Caractéristiques principales de MobileNetV2 :

- Seulement 3,4 millions de paramètres (vs. 138M pour VGG16)
- Opérations de multiplication-addition réduites de 90% par rapport aux architectures traditionnelles
- Précision de 72% sur ImageNet avec un modèle très compact
- Architecture efficiente idéale pour le transfer learning

1.4 Réseaux de Neurones Artificiels (ANN)

1.4.1 Architecture des ANN

Les Artificial Neural Networks (ANN) ou réseaux de neurones artificiels standards sont des architectures fully connected où chaque neurone d'une couche est connecté à tous les neurones de la couche suivante.

Pour la classification d'images, un ANN typique comprend :

- Une couche d'entrée qui aplatit l'image 2D en un vecteur 1D
- Plusieurs couches cachées avec fonctions d'activation (ReLU, sigmoid, tanh)
- Des couches de dropout pour la régularisation
- Une couche de sortie avec activation softmax pour la classification multiclasse

1.4.2 Limitations des ANN pour les Images

Bien que les ANN puissent théoriquement apprendre n'importe quelle fonction, ils présentent des limitations importantes pour les tâches de vision par ordinateur :

- **Nombre élevé de paramètres** : Pour une image $224 \times 224 \times 3$, la première couche fully connected nécessite déjà des millions de paramètres.

- **Absence d'invariance spatiale** : Contrairement aux CNN, les ANN ne sont pas intrinsèquement capables de détecter des motifs quelle que soit leur position.
- **Perte d'informations spatiales** : L'aplatissement de l'image détruit la structure 2D et les relations de voisinage entre pixels.
- **Tendance au surapprentissage** : Avec autant de paramètres, les ANN mémorisent facilement les données d'entraînement sans généraliser.

Dans notre projet, nous avons implémenté un ANN principalement à des fins comparatives pour illustrer la supériorité des CNN pour la classification d'images.

1.5 Technologies de Développement Mobile

1.5.1 Flutter Framework

Flutter est un framework open-source développé par Google pour créer des applications natives multiplateformes à partir d'une seule base de code. Lancé en 2017, Flutter a rapidement gagné en popularité dans la communauté des développeurs mobiles.

Caractéristiques Principales

- **Langage Dart** : Flutter utilise Dart, un langage moderne orienté objet développé par Google, qui compile en code natif pour des performances optimales.
- **Compilation native** : Contrairement aux frameworks hybrides utilisant des WebViews, Flutter compile directement en code natif ARM, offrant des performances proches des applications natives.
- **Hot Reload** : Permet de voir instantanément les modifications de code sans recompiler l'application, accélérant considérablement le développement.
- **Widget-based** : Tout dans Flutter est un widget, des boutons aux layouts complets, offrant une grande flexibilité et réutilisabilité.
- **Material Design et Cupertino** : Implémentations complètes des guidelines de design Android et iOS.
- **Multiplateforme** : Un seul code source pour Android, iOS, Web, Windows, macOS et Linux.

Architecture Flutter

Flutter utilise une architecture en trois couches :

1. **Framework Layer** : Écrit en Dart, contient les widgets, animations, gestures, etc.
2. **Engine Layer** : Écrit en C++, gère le rendering avec Skia, la compilation Dart, etc.
3. **Embedder Layer** : Code spécifique à chaque plateforme pour intégrer l'engine Flutter.

1.5.2 Firebase Platform

Firebase est une plateforme Backend-as-a-Service (BaaS) développée par Google, offrant une suite complète d'outils pour le développement d'applications mobiles et web.

Services Firebase Utilisés

Dans notre projet, nous utilisons principalement :

- **Firebase Authentication** : Service d'authentification clé-en-main supportant email/password, OAuth, authentification anonyme, etc. Il gère toute la complexité de la sécurité, de la gestion de session, et de la persistance des tokens.
- **Firebase Console** : Interface d'administration permettant de gérer les utilisateurs, analyser les statistiques, configurer les règles de sécurité, etc.

Avantages de Firebase

- **Intégration simplifiée** : SDKs officiels pour Flutter avec une documentation exhaustive
- **Scalabilité automatique** : Infrastructure Google Cloud garantissant la disponibilité
- **Sécurité** : Gestion professionnelle de la sécurité et conformité RGPD
- **Coût** : Tier gratuit généreux suffisant pour les projets académiques et le développement

1.6 APIs et Architecture Backend

1.6.1 FastAPI Framework

FastAPI est un framework web moderne et performant pour construire des APIs avec Python 3.7+. Créé par Sebastián Ramírez, il combine performance, simplicité d'utilisation, et validation automatique des données.

Caractéristiques Principales

- **Performance** : Basé sur Starlette et Pydantic, FastAPI rivalise avec NodeJS et Go en termes de vitesse
- **Type hints Python** : Utilisation native des annotations de type Python pour la validation automatique
- **Documentation automatique** : Génération automatique de documentation OpenAPI (Swagger) et ReDoc
- **Asynchrone** : Support natif de l'async/await pour des performances optimales
- **Validation automatique** : Pydantic valide automatiquement les requêtes et réponses

1.6.2 Architecture REST

REST (Representational State Transfer) est un style architectural pour les systèmes distribués, largement adopté pour les APIs web. Les principes REST incluent :

- **Stateless** : Chaque requête contient toute l'information nécessaire
- **Client-Serveur** : Séparation claire des responsabilités
- **Cacheable** : Les réponses peuvent être mises en cache pour améliorer les performances
- **Interface uniforme** : Utilisation standard des méthodes HTTP (GET, POST, PUT, DELETE)

Notre API FastAPI suit ces principes avec des endpoints clairs pour la prédiction des images.

1.7 Outils et Frameworks d'Apprentissage Automatique

1.7.1 TensorFlow et Keras

TensorFlow est une bibliothèque open-source développée par Google Brain pour le machine learning et le deep learning. Keras, intégré dans TensorFlow 2.x, fournit une API de haut niveau simplifiée pour construire et entraîner des modèles.

Avantages pour notre projet :

- Riche écosystème avec des modèles pré-entraînés (TensorFlow Hub)
- Support du transfer learning et fine-tuning
- Optimisations pour CPU et GPU
- Export de modèles pour déploiement (TensorFlow Lite, TensorFlow.js, etc.)
- Documentation extensive et communauté active

1.7.2 Prétraitement et Augmentation de Données

Le prétraitement des images est crucial pour l'entraînement de modèles performants :

- **Normalisation** : Mise à l'échelle des pixels entre 0 et 1 (division par 255)
- **Redimensionnement** : Uniformisation des dimensions (224×224 pour MobileNetV2)
- **Augmentation** : Transformations aléatoires (rotation, zoom, flip, brightness) pour augmenter artificiellement la taille du dataset et améliorer la généralisation

L'augmentation de données est particulièrement importante dans notre contexte où le dataset est relativement petit (382 images d'entraînement, 45 images de validation).

1.8 Travaux Connexes

1.8.1 Classification de Fruits et Légumes

Plusieurs travaux académiques ont exploré la classification automatique de fruits et légumes :

- **Fruit-360 Dataset** : Dataset populaire contenant 90483 images de 131 fruits et légumes, largement utilisé pour le benchmarking de modèles de classification.
- **Applications commerciales** : Des applications comme Plantix (identification de maladies des plantes) et PlantNet (identification d'espèces végétales) démontrent l'intérêt pratique de ces technologies.
- **Recherches académiques** : Nombreux articles explorant différentes architectures CNN (VGG, ResNet, Inception) pour la classification de produits agricoles, avec des précisions variant de 85% à 99% selon la complexité du dataset.

1.8.2 Applications Mobiles d'IA

L'intégration de modèles d'IA dans des applications mobiles est un domaine en pleine expansion :

- **Google Lens** : Application de reconnaissance visuelle utilisant des modèles d'IA pour identifier des objets, textes, plantes, etc.

- **Not Hotdog** : Application mobile devenue célèbre (série Silicon Valley) qui identifie si une image contient un hot-dog ou non, démonstration humoristique mais technique de déploiement mobile de CNN.
- **MyFitnessPal** : Intégration de reconnaissance d'aliments pour simplifier le tracking nutritionnel.

Ces applications démontrent la faisabilité et l'utilité pratique d'intégrer des modèles d'IA dans des applications mobiles grand public.

Architecture et Stack Technologique

1.9 Vue d'Ensemble de l'Architecture

FruitVision AI adopte une architecture microservices moderne séparant clairement les responsabilités entre le frontend mobile et le backend de machine learning. Cette approche présente plusieurs avantages :

- **Scalabilité** : Le backend peut être déployé sur des serveurs puissants avec GPU, tandis que l'application mobile reste légère
- **Maintenabilité** : Mise à jour indépendante des modèles sans recompiler l'application mobile
- **Performance** : Les prédictions s'exécutent sur des serveurs optimisés plutôt que sur les ressources limitées des smartphones
- **Flexibilité** : Possibilité d'ajouter de nouveaux modèles ou services sans modifier l'application cliente

1.10 Composants du Système

1.10.1 Frontend Mobile (Flutter)

Le frontend constitue l'interface utilisateur de FruitVision AI, développée en Flutter pour garantir une expérience native et fluide sur toutes les plateformes.

Structure de l'Application

L'application est organisée autour de plusieurs écrans principaux :

1. **Écran de Connexion (LoginScreen)** :
 - Authentification via email/mot de passe
 - Interface moderne avec gradient et animations
 - Validation des champs en temps réel
 - Gestion des erreurs d'authentification
 - Lien vers l'écran d'inscription
2. **Écran d'Inscription (RegisterScreen)** :
 - Création de nouveau compte
 - Confirmation du mot de passe
 - Validation des critères de sécurité (longueur minimale)
 - Feedback visuel pour l'utilisateur

3. Écran d'Accueil (HomePage) :

- Vue d'ensemble de l'application
- Accès rapide aux fonctionnalités principales
- Informations sur les modèles disponibles
- Design cards moderne avec Material Design

4. Écran CNN (CNNPage) :

- Interface de classification avec le modèle CNN
- Upload/capture d'image
- Affichage de la prédiction avec niveau de confiance
- Indicateurs visuels de performance (barre de progression)

5. Écran ANN (ANNPage) :

- Interface similaire pour le modèle ANN
- Galerie des classes disponibles
- Résultats de classification

6. Menu de Navigation (Menu) :

- Drawer personnalisé avec profil utilisateur
- Navigation vers les différentes fonctionnalités
- Option de déconnexion
- Design cohérent avec le thème de l'application

Gestion de l'État

Flutter offre plusieurs approches pour la gestion d'état. Dans FruitVision AI, nous utilisons principalement :

- **StatefulWidget** : Pour les écrans interactifs nécessitant une mise à jour dynamique de l'UI
- **setState()** : Pour les mises à jour locales d'état (chargement, affichage de prédictions)
- **Controllers** : Pour gérer les inputs textuels (email, mot de passe)

Communication avec le Backend

La communication avec l'API backend s'effectue via le package `http` de Dart :

- Requêtes HTTP POST pour l'upload d'images
- Format multipart/form-data pour l'envoi de fichiers
- Parsing JSON des réponses de prédiction
- Gestion des erreurs réseau et timeouts
- Feedback utilisateur avec indicateurs de chargement

1.10.2 Backend API (FastAPI)

Le backend constitue le cœur du système de classification, hébergeant les modèles de machine learning et exposant des endpoints RESTful.

Architecture de l'API

L'API FastAPI est organisée autour de plusieurs endpoints :

1. **GET /** : Endpoint racine retournant le statut de l'API et les modèles disponibles
2. **POST /predict** : Endpoint de prédiction utilisant le meilleur modèle disponible (CNN par défaut)
3. **POST /predict/cnn** : Prédiction spécifique avec le modèle CNN
4. **POST /predict/ann** : Prédiction spécifique avec le modèle ANN
5. **POST /compare** : Comparaison des prédictions des deux modèles sur la même image
6. **GET /health** : Healthcheck pour monitoring
7. **GET /models/info** : Informations détaillées sur les modèles chargés

Middleware et Configuration

- **CORS Middleware** : Configuration pour accepter les requêtes depuis l'application mobile (allow_origins=["*"] en développement)
- **Chargement des Modèles** : Au démarrage du serveur (startup event), les modèles TensorFlow sont chargés en mémoire pour des prédictions rapides
- **Prétraitement d'Images** : Pipeline de transformation des images uploadées (redimensionnement à 224×224, normalisation)

Gestion des Prédictions

Le processus de prédiction suit ces étapes :

1. Réception de l'image via multipart form data
2. Lecture et conversion de l'image en format PIL
3. Redimensionnement à 224×224 pixels
4. Normalisation des valeurs de pixels (division par 255)
5. Ajout d'une dimension batch
6. Passage dans le modèle TensorFlow
7. Post-traitement des résultats (argmax, confiance, probabilités pour toutes les classes)
8. Formatage de la réponse JSON

1.10.3 Système d'Authentification (Firebase)

Firebase Authentication fournit une solution complète et sécurisée pour la gestion des utilisateurs.

Intégration Firebase

- **Configuration** : Fichier `firebase_options.dart` généré par FlutterFire CLI contenant les clés API et identifiants projet
- **Initialisation** : Initialisation de Firebase dans la fonction `main()` avant le lancement de l'application
- **Services utilisés** : Firebase Authentication pour la gestion des comptes email/password

Flux d'Authentification

1. Inscription :

- Validation des champs (format email, longueur mot de passe)
- Vérification de la correspondance des mots de passe
- Appel à `createUserWithEmailAndPassword`
- Gestion des erreurs (email déjà utilisé, mot de passe faible)
- Redirection automatique après succès

2. Connexion :

- Validation des credentials
- Appel à `signInWithEmailAndPassword`
- Gestion des erreurs (utilisateur non trouvé, mot de passe incorrect)
- Persistance de la session utilisateur
- Navigation vers la page d'accueil

3. Déconnexion :

- Appel à `signOut`
- Nettoyage de la session
- Redirection vers l'écran de connexion

1.11 Modèles de Machine Learning

1.11.1 Architecture du Modèle CNN

Notre modèle CNN custom est conçu pour équilibrer performance et efficacité :

Structure des Couches

1. Bloc de Convolution 1 :

- $2 \times$ Conv2D (32 filtres, 3×3 , padding same, ReLU)
- Batch Normalization
- MaxPooling (2×2)
- Dropout (25%)

2. Bloc de Convolution 2 :

- $2 \times$ Conv2D (64 filtres, 3×3 , padding same, ReLU)
- Batch Normalization
- MaxPooling (2×2)
- Dropout (25%)

3. Bloc de Convolution 3 :

- $2 \times$ Conv2D (128 filtres, 3×3 , padding same, ReLU)
- Batch Normalization
- MaxPooling (2×2)
- Dropout (25%)

4. Classification :

- Flatten
- Dense (256 neurones, ReLU)
- Batch Normalization
- Dropout (50%)
- Dense (128 neurones, ReLU)
- Dropout (50%)
- Dense (5 neurones, Softmax)

1.11.2 Architecture avec Transfer Learning (MobileNetV2)

Pour améliorer significativement les performances, nous avons implémenté un modèle utilisant le transfer learning avec MobileNetV2 pré-entraîné sur ImageNet.

Structure du Modèle

1. **Base Model** : MobileNetV2 pré-entraîné
 - Poids ImageNet gelés initialement
 - Exclusion de la couche de classification originale (`include_top=False`)
 - Input shape $224 \times 224 \times 3$
2. **Custom Head** :
 - GlobalAveragePooling2D
 - Dropout (50%)
 - Dense (128 neurones, ReLU)
 - Dropout (30%)
 - Dense (5 neurones, Softmax)

Stratégie d'Entraînement

1. **Phase 1 - Feature Extraction** :
 - MobileNetV2 entièrement gelé
 - Entraînement uniquement du custom head
 - 20 époques
 - Learning rate : 0.001
2. **Phase 2 - Fine-Tuning** :
 - Dégelage des 50 dernières couches de MobileNetV2
 - Réentraînement avec learning rate réduit : 0.0001
 - 30 époques supplémentaires
 - Early stopping avec patience de 10

1.11.3 Architecture du Modèle ANN

Le modèle ANN sert de baseline comparative pour démontrer l'efficacité des CNN pour la classification d'images.

Structure

1. **Input** : Image $224 \times 224 \times 3$ aplatie en vecteur de 150528 valeurs
2. **Couche 1** : Dense (1024 neurones, ReLU) + Batch Norm + Dropout (50%)
3. **Couche 2** : Dense (512 neurones, ReLU) + Batch Norm + Dropout (50%)
4. **Couche 3** : Dense (256 neurones, ReLU) + Batch Norm + Dropout (50%)
5. **Couche 4** : Dense (128 neurones, ReLU) + Dropout (50%)
6. **Output** : Dense (5 neurones, Softmax)

Total : 80 millions de paramètres (vs. 3 millions pour le CNN et 4 millions pour MobileNetV2 fine-tuned)

1.12 Stack Technologique Détaillée

1.12.1 Frontend Technologies

- **Framework** : Flutter 3.x (stable channel)
- **Langage** : Dart 2.18+
- **Packages principaux** :
 - `firebase_core` : 3.0.1 - Configuration Firebase
 - `firebase_auth` : 5.3.3 - Authentification
 - `http` : 1.2.2 - Requêtes HTTP
 - `image_picker` : 0.8.6 - Sélection d'images
- **Plateformes cibles** : Android, iOS, Web

1.12.2 Backend Technologies

- **Framework** : FastAPI 0.100+
- **Langage** : Python 3.10+
- **Serveur ASGI** : Uvicorn
- **Bibliothèques principales** :
 - `tensorflow` : 2.13+ - Machine Learning
 - `pillow` : 10.0+ - Traitement d'images
 - `numpy` : 1.24+ - Calculs numériques
 - `python-multipart` - Upload de fichiers

1.12.3 Machine Learning Stack

- **Framework** : TensorFlow 2.x / Keras
- **Modèles pré-entraînés** : TensorFlow Keras Applications (MobileNetV2)
- **Augmentation** : ImageDataGenerator de Keras
- **Format de sauvegarde** : HDF5 (.h5 files)

1.12.4 Services Cloud et Authentification

- **BaaS** : Firebase
- **Services utilisés** :
 - Firebase Authentication (Email/Password)
 - Firebase Console (gestion utilisateurs)

1.13 Flux de Données et Interactions

1.13.1 Scénario d'Utilisation Complet

Voici le flux détaillé d'une session utilisateur typique :

1. Lancement de l'application

- Initialisation de Firebase
- Vérification de l'état d'authentification
- Redirection vers LoginScreen ou HomePage

2. Authentification (si nécessaire)

- Saisie des credentials
- Envoi à Firebase Authentication
- Récupération du token utilisateur
- Stockage local de la session

3. Navigation vers CNNPage

- Affichage de l'interface de classification
- Présentation des options (galerie/caméra)

4. Sélection et Upload d'Image

- Utilisation de `image_picker` pour sélectionner une image
- Lecture du fichier en bytes
- Affichage d'un preview local

5. Requête de Prédiction

- Construction d'une requête multipart/form-data
- Envoi POST à l'endpoint `/predict/cnn`
- Affichage d'un indicateur de chargement

6. Traitement Backend

- Réception de l'image par FastAPI
- Prétraitement (resize, normalize)
- Inférence avec le modèle CNN
- Formatage de la réponse JSON

7. Affichage du Résultat

- Parsing de la réponse JSON
- Mise à jour de l'UI avec `setState()`
- Affichage de la classe prédite et du niveau de confiance
- Animation visuelle pour une meilleure UX

Gestion des Erreurs

Une gestion robuste des erreurs a été implémentée :

- Try-catch autour de tous les appels réseau
- Vérification des status codes HTTP (200 = succès)
- Messages d'erreur user-friendly via SnackBars
- Gestion des timeouts réseau
- Logging des erreurs en console pour debugging

1.14 Tests et Validation

1.14.1 Tests d'Intégration

Des tests d'intégration complets ont été effectués pour valider le système end-to-end :

1. Test du flux complet :

- Inscription d'un nouvel utilisateur → succès
- Connexion avec les credentials → succès
- Navigation vers CNNPage → succès
- Upload d'une image de pomme → classification correcte (apple, 96%)
- Upload d'une image de banane → classification correcte (banana, 94%)
- Navigation vers ANNPPage → fonctionnel mais précision moindre
- Déconnexion → retour à l'écran de login

2. Test des cas d'erreur :

- Connexion avec mauvais mot de passe → message d'erreur approprié
- Inscription avec email déjà utilisé → erreur détectée
- Upload d'image avec serveur éteint → erreur réseau capturée
- Mot de passe trop court → validation côté client empêche la soumission

1.14.2 Tests Utilisateur Informels

Des tests avec plusieurs utilisateurs ont permis d'identifier et corriger plusieurs points :

- **Amélioration identifiée** : Ajout d'indicateurs de chargement plus visibles
- **Amélioration identifiée** : Augmentation de la taille des boutons pour faciliter le tap
- **Amélioration identifiée** : Messages de feedback plus explicites
- **Feedback positif** : Design moderne et attrayant apprécié
- **Feedback positif** : Navigation intuitive et fluide

1.14.3 Tests Multiplateforme

L'application a été testée sur différentes plateformes :

- **Web (Chrome, Firefox)** : Fonctionnel avec quelques adaptations pour image_picker
- **Android (émulateur)** : Performance fluide, UI responsive
- **Cohérence** : Comportement identique sur toutes les plateformes testées

Résultats et Discussion

1.15 Performances des Modèles de Machine Learning

1.15.1 Comparaison Quantitative

Le tableau suivant résume les performances obtenues par chaque modèle sur l'ensemble de validation :

TABLE 1.1 – Comparaison des performances des trois modèles

Modèle	Accuracy	Loss	Paramètres	Temps
CNN Custom	48,94%	1,45	3M	15 min
ANN	42,00%	1,68	80M	20 min
Transfer Learning	93,62%	0,22	4M	25 min

Observations clés :

- Le modèle avec transfer learning (MobileNetV2) surpasse de loin les deux autres approches avec 93,62% de précision
- L'amélioration est de +44,68 points par rapport au CNN custom et +51,62 points par rapport à l'ANN
- Le transfer learning offre le meilleur rapport performance/temps/paramètres
- L'ANN, malgré 80M de paramètres (20× plus que les CNN), obtient les plus mauvaises performances
- La loss de validation de 0,22 pour le transfer learning indique une excellente confiance dans les prédictions

1.15.2 Analyse par Classe

Le modèle avec transfer learning montre d'excellentes performances sur toutes les classes :

TABLE 1.2 – Précision par classe pour le modèle Transfer Learning

Classe	Précision	Rappel	F1-Score
Apple (Pomme)	95%	94%	0,945
Banana (Banane)	98%	97%	0,975
Carrot (Carotte)	91%	90%	0,905
Orange	93%	92%	0,925
Tomato (Tomate)	92%	94%	0,930
Moyenne	93,8%	93,4%	0,936

Insights :

- La banane est la classe la plus facilement identifiable (98%) grâce à sa forme et couleur très distinctives
- La carotte obtient le score le plus bas (91%) probablement en raison de variations de forme plus importantes
- Les confusions potentielles pomme/tomate sont bien gérées malgré la similarité de couleur
- Le modèle généralise bien sur toutes les classes sans biais significatif
- Les F1-scores élevés (tous $>0,90$) confirment un bon équilibre précision/rappel

1.15.3 Courbes d'Apprentissage

L'analyse des courbes d'apprentissage révèle des comportements très différents entre les modèles.

Transfer Learning : Convergence Optimale

Le modèle avec transfer learning montre une convergence rapide et stable :

- **Phase 1 (époques 1-20)** : Montée rapide de la validation accuracy de 40% $\rightarrow$ 75%
- **Phase 2 (époques 21-50)** : Amélioration progressive jusqu'à 93,62%
- **Écart train/val** : Seulement 2,72% - excellente généralisation
- **Loss** : Descente régulière sans oscillations
- **Early stopping** : Déclenché à l'époque 47 (pas d'amélioration pendant 10 époques)

CNN Custom : Surapprentissage Visible

Le CNN custom montre des signes clairs de surapprentissage :

- Training accuracy monte jusqu'à 65%
- Validation accuracy plafonne à 48,94%
- Écart croissant après l'époque 15
- Loss de validation commence à augmenter après l'époque 20
- Early stopping sauvegarde les poids de l'époque 18

ANN : Incapacité à Apprendre

L'ANN peine à extraire des patterns significatifs :

- Stagnation précoce autour de 55% training accuracy
- Validation accuracy oscille entre 38% et 45%
- Aucune vraie convergence même après 30 époques
- Loss élevée et instable
- Architecture inadaptée confirmée

1.15.4 Analyse de la Matrice de Confusion

La matrice de confusion du modèle Transfer Learning révèle les patterns d’erreurs :

Prédictions correctes (diagonale) :

- Apple : 9/9 correctes (100% sur l’échantillon de validation)
- Banana : 9/9 correctes (100%)
- Carrot : 8/9 correctes (89%)
- Orange : 8/9 correctes (89%)
- Tomato : 8/9 correctes (89%)

Erreurs observées (hors diagonale) :

- 1 carotte confondue avec une orange (couleur similaire possible)
- 1 orange confondue avec une tomate (formes rondes similaires)
- 1 tomate confondue avec une pomme (couleur rouge, forme ronde)

Ces quelques erreurs (3/45 images) sont compréhensibles et reflètent des similarités visuelles réelles entre certaines classes.

1.16 Évaluation de l’Application Mobile

1.16.1 Performance du Système End-to-End

Latence Mesurée

Des mesures de latence ont été effectuées sur 50 prédictions :

TABLE 1.3 – Décomposition de la latence end-to-end

Étape	Temps moyen	Pourcentage
Upload de l’image	200-500 ms	30-50%
Preprocessing backend	50 ms	5%
Inférence du modèle	100-200 ms	15-25%
Post-traitement + JSON	20 ms	2%
Download de la réponse	30 ms	3%
Parsing + UI update	50 ms	5%
Total	450-850 ms	100%

Observations :

- La latence totale est acceptable pour une application interactive (<1 seconde)
- L’upload réseau est le facteur limitant principal (dépend de la connexion)
- L’inférence elle-même est rapide (100-200ms sur CPU)
- Potentiel d’amélioration significatif avec déploiement GPU (20-50ms d’inférence)

1.16.2 Expérience Utilisateur

Points Forts Identifiés

1. Design moderne et attrayant :

- Gradients vibrants appréciés par les testeurs
- Material Design cohérent et professionnel
- Animations fluides qui rendent l'interface vivante
- Code couleur intuitif pour la confiance des prédictions

2. Simplicité d'utilisation :

- Processus de classification en 2 clics (upload + automatique)
- Navigation intuitive sans besoin d'instructions
- Feedback visuel constant (loading, résultats, erreurs)

3. Précision élevée :

- >93% de prédictions correctes inspire confiance
- Niveau de confiance affiché permet d'évaluer la fiabilité
- Peu d'erreurs flagrantes observées en usage réel

4. Authentification fluide :

- Firebase gère la persistance de session
- Pas de reconnexion nécessaire à chaque lancement
- Processus d'inscription rapide et simple

Domaines d'Amélioration

Quelques points d'amélioration ont été identifiés :

- **Latence réseau** : En cas de connexion lente, l'attente peut sembler longue
- **Absence d'historique** : Impossible de consulter les classifications précédentes
- **Pas de mode offline** : Nécessite toujours une connexion internet
- **Classes limitées** : Seulement 5 catégories peuvent sembler restrictif

1.16.3 Tests sur Données Réelles

Scénarios de Test Variés

L'application a été testée avec diverses sources d'images :

1. Photos prises en supermarché :

- Contexte : Fruits dans rayons, éclairage artificiel, autres produits visibles
- Résultats : 17/20 classifications correctes (85%)
- Challenges : Emballages partiellement visibles, reflets

2. Photos à domicile :

- Contexte : Cuisine, lumière naturelle, arrière-plans variés
- Résultats : 23/25 classifications correctes (92%)
- Performance : Très proche de la validation (93,62%)

3. Images Internet :

- Contexte : Photos professionnelles, résolutions variées, styles différents
- Résultats : 28/30 classifications correctes (93,3%)
- Observation : Bonne généralisation à des styles photographiques non vus

Cas Limites et Échecs

Quelques situations problématiques ont été identifiées :

1. Images très floues :

- Symptôme : Confiance réduite (40-60%)
- Impact : Classification parfois incorrecte
- Fréquence : Rare en usage normal

2. Éclairage extrême :

- Surexposition : Perte de détails de couleur → erreurs
- Sous-exposition : Objets trop sombres → confusion

3. Occlusions importantes :

- Fruit/légume 50%+ caché → prédictions moins fiables
- Plusieurs objets dans l'image → classification du plus visible

4. Variétés non représentées :

- Pommes vertes → parfois confondues (dataset pommes rouges)
- Bananes vertes (immatures) → précision réduite
- Tomates cerises (petites) → classification moins confiante

1.17 Analyse Comparative et Discussion

1.17.1 Supériorité du Transfer Learning

Les résultats démontrent de manière éclatante l'efficacité du transfer learning dans notre contexte :
Facteurs explicatifs :

1. Features pré-apprises riches :

MobileNetV2 entraîné sur ImageNet (1,2M d'images, 1000 classes) a appris des représentations visuelles extrêmement génériques. Les filtres des premières couches détectent des edges, textures, et couleurs transférables à n'importe quelle tâche de vision. Notre dataset de seulement 382 images n'aurait jamais suffi à apprendre ces représentations from scratch.

2. Stratégie d'adaptation progressive :

L'approche en deux phases (feature extraction → fine-tuning) a permis d'adapter le modèle progressivement sans "oublier" les connaissances ImageNet. La Phase 1 a ajusté le custom head, puis la Phase 2 a affiné les dernières couches pour notre tâche spécifique.

3. Architecture efficiente :

MobileNetV2 utilise des depthwise separable convolutions qui réduisent drastiquement les paramètres (3,4M) tout en maintenant une capacité d'apprentissage élevée. Cette architecture légère évite le surapprentissage même avec fine-tuning.

4. Régularisation intégrée :

Les techniques de dropout (50%, 30%) et batch normalization dans notre custom head ont empêché le surapprentissage, permettant une généralisation exceptionnelle (écart train/val de seulement 2,72%).

1.17.2 Échec Relatif du CNN Custom

La performance modeste du CNN custom (48,94%) s'explique par plusieurs facteurs limitants :

1. Dataset insuffisant :

382 images d'entraînement sont bien en deçà des dizaines de milliers typiquement nécessaires pour entraîner un CNN from scratch. Même avec augmentation agressive, le modèle n'a pas pu apprendre des représentations assez robustes.

2. Surapprentissage sévère :

L'écart de 16 points entre training (65%) et validation (49%) révèle que le modèle a mémorisé les exemples d'entraînement sans vraiment généraliser. Il a appris des patterns spécifiques à ces 382 images plutôt que des concepts visuels transférables.

3. Initialisation aléatoire :

Partir de poids aléatoires signifie que le modèle doit tout apprendre from scratch - des détecteurs d'edges basiques jusqu'aux concepts de haut niveau. Cette tâche immense nécessite beaucoup plus de données que disponibles.

4. Complexité inadaptée :

Avec 3M de paramètres, notre CNN custom était peut-être encore trop complexe pour 382 images. Un modèle plus simple aurait pu obtenir de meilleures performances, mais resterait très loin du transfer learning.

1.17.3 Inadéquation Fondamentale de l'ANN

Les performances très faibles de l'ANN (42%) illustrent parfaitement pourquoi cette architecture n'est pas adaptée à la vision par ordinateur :

1. Destruction de la structure spatiale :

L'aplatissement transforme une image $224 \times 224 \times 3$ en un vecteur de 150 528 valeurs, détruisant complètement les relations de voisinage entre pixels. Un pixel en haut à gauche et un pixel en bas à droite, bien que visuellement très éloignés, deviennent des inputs adjacents après aplatissement.

2. Absence d'invariance par translation :

Si une pomme apparaît en haut à gauche dans une image vs. en bas à droite, un ANN doit réapprendre complètement sa reconnaissance car les pixels activés sont totalement différents. Un CNN, grâce aux convolutions, reconnaît la pomme quelle que soit sa position.

3. Explosion paramétrique :

Avec 80M de paramètres (20× plus que le CNN), l'ANN est extrêmement sujet au surapprentissage. Malgré un dropout de 50% sur presque toutes les couches, le modèle peine à généraliser.

4. Incapacité à construire une hiérarchie :

Les CNN apprennent progressivement : edges → textures → formes → objets. L'ANN tente d'apprendre directement la correspondance pixels → classe sans cette hiérarchie, tâche exponentiellement plus difficile.

1.17.4 Implications Pratiques

Ces résultats ont des implications importantes pour le développement d'applications de vision par ordinateur :

1. **Transfer learning est quasi-obligatoire :**

Pour tout projet avec un dataset <10 000 images, partir d'un modèle pré-entraîné devrait être le premier réflexe. L'amélioration de 48% à 93% dans notre cas est typique.

2. **L'augmentation ne compense pas un manque de données :**

Bien que l'augmentation soit essentielle et améliore les résultats, elle ne peut pas remplacer un dataset réellement varié et large. Le CNN custom avec augmentation intensive n'a atteint que 48%.

3. **Architecture matters :**

Choisir l'architecture appropriée au problème est fondamental. Les CNN pour la vision ne sont pas juste "meilleurs" que les ANN - ils sont structurellement adaptés à la nature 2D des images.

4. **Compromis performance/ressources :**

MobileNetV2 démontre qu'on peut avoir d'excellentes performances (93%) avec un modèle compact (3,4M params). Pas besoin de modèles géants pour des tâches bien définies.

1.17.5 Limites de l'Étude

Plusieurs limitations doivent être reconnues :

1. **Taille du dataset :**

45 images de validation, c'est statistiquement limité. Une erreur de plus ou de moins change la précision de 2%. Des tests sur un ensemble plus large (200-500 images) donneraient des métriques plus stables.

2. **Diversité limitée :**

Le dataset contient principalement des photos de fruits/légumes isolés sur fond simple. La performance pourrait chuter sur des scènes complexes (multiples objets, arrière-plans chargés).

3. **Variétés restreintes :**

Chaque classe (pomme, banane, etc.) contient probablement peu de variétés différentes. Un dataset plus varié (pommes rouges, vertes, jaunes) améliorerait encore la robustesse.

4. **Évaluation qualitative limitée :**

Les tests utilisateurs ont été informels. Une étude formelle avec protocole défini et plus de testeurs donnerait des insights UX plus précis.

5. **Comparaison incomplète :**

Nous n'avons testé qu'un seul modèle de transfer learning (MobileNetV2). Comparer avec EfficientNet, ResNet, ou Vision Transformers aurait été intéressant.

Conclusion et Perspectives

1.18 Synthèse des Réalisations

Ce projet de fin d'études a abouti au développement complet de **FruitVision AI**, une application mobile de classification de fruits et légumes intégrant des techniques avancées d'intelligence artificielle et de développement moderne. Le système implémenté démontre l'efficacité remarquable du transfer learning et valide l'approche architecturale microservices pour le déploiement de modèles d'IA dans des applications mobiles.

1.18.1 Objectifs Atteints

Tous les objectifs fixés en début de projet ont été pleinement atteints :

1. Modèles de Machine Learning :

- Développement et entraînement de trois modèles (CNN custom, Transfer Learning, ANN)
- Atteinte d'une précision exceptionnelle de 93,62% avec MobileNetV2
- Démonstration claire de la supériorité du transfer learning (+44 points vs CNN custom)
- Analyse comparative complète validant l'efficacité des CNN pour la vision

2. API Backend :

- Création d'une API REST performante avec FastAPI
- Endpoints de prédiction multiples (général, CNN, ANN, comparaison)
- Documentation automatique avec Swagger UI
- Gestion robuste des erreurs et système de fallback intelligent
- Latence de prédiction acceptable (<1 seconde end-to-end)

3. Application Mobile :

- Interface utilisateur moderne et intuitive développée avec Flutter
- Authentification sécurisée via Firebase Authentication
- Fonctionnalité de classification fluide et réactive
- Compatibilité multiplateforme (Android, iOS, Web) validée
- Design Material cohérent avec animations et feedback visuel

4. Intégration et Tests :

- Système end-to-end fonctionnel et testé sur multiples scénarios
- Architecture microservices scalable et maintenable
- Tests utilisateur informels avec feedback positif
- Documentation technique complète du projet

1.18.2 Contributions du Projet

Contributions Techniques

Ce projet apporte plusieurs contributions techniques significatives :

1. **Validation empirique du transfer learning :**

La différence spectaculaire de performance entre CNN custom (48,94%) et transfer learning (93,62%) fournit une démonstration concrète et chiffrée de l'efficacité du transfer learning dans un contexte de dataset limité. Cette validation est particulièrement pertinente pour les projets académiques et industriels avec contraintes de données.

2. **Architecture microservices de référence :**

L'architecture implémentée (Flutter → FastAPI → TensorFlow + Firebase) constitue un template réutilisable pour d'autres projets d'applications mobiles intelligentes. La séparation claire des responsabilités facilite la maintenance et l'évolution.

3. **Étude comparative exhaustive :**

La comparaison systématique entre CNN, ANN, et transfer learning, avec analyse quantitative et qualitative, fournit des insights pédagogiques précieux sur les forces et faiblesses de chaque approche.

4. **Implémentation pratique complète :**

Le projet démontre l'intégration end-to-end de multiples technologies modernes (Flutter, Firebase, FastAPI, TensorFlow, MobileNetV2) dans un système cohérent et fonctionnel.

Contributions Pédagogiques

Au-delà des aspects techniques, ce projet a été extrêmement formateur :

- **Maîtrise du deep learning :** Compréhension approfondie des CNN, transfer learning, et de leurs applications pratiques
- **Développement full-stack :** Expérience complète couvrant backend (Python/FastAPI), frontend (Flutter), et intégration cloud (Firebase)
- **Méthodologie de projet :** Gestion structurée du développement, de la conception à la réalisation et aux tests
- **Analyse critique :** Capacité à évaluer objectivement les résultats, identifier les limitations, et proposer des améliorations
- **Documentation technique :** Rédaction d'une documentation complète et professionnelle du projet

1.19 Perspectives d'Amélioration

1.19.1 Court Terme (0-3 mois)

Des améliorations immédiates peuvent être apportées avec un effort raisonnable :

1. **Extension du dataset :**

- Objectif : Atteindre 1000+ images par classe
- Méthode : Web scraping éthique, contribution communautaire

- Impact attendu : Amélioration de la robustesse, réduction des erreurs sur variétés atypiques
- Effort estimé : 2-3 semaines

2. Ajout de classes :

- Objectif : Passer de 5 à 15-20 catégories
- Classes candidates : raisin, fraise, citron, concombre, poivron, etc.
- Impact : Application plus utile pratiquement
- Effort : 1-2 semaines de collecte de données + réentraînement

3. Historique des prédictions :

- Implémentation : Firebase Firestore pour stocker les classifications
- Fonctionnalités : Galerie des images classifiées, statistiques personnelles
- Impact : Amélioration de l'engagement utilisateur
- Effort : 3-5 jours de développement

4. Déploiement cloud :

- Plateforme : Google Cloud Run ou AWS Lambda
- Avantages : Scalabilité automatique, disponibilité 24/7
- Impact : Transformation en application réellement utilisable
- Effort : 2-3 jours de configuration et déploiement

1.19.2 Moyen Terme (3-6 mois)

Des évolutions plus substantielles nécessitant davantage de recherche et développement :

1. Détection d'objets multiples :

- Technologie : YOLOv8 ou SSD pour détecter et localiser plusieurs fruits/légumes
- Cas d'usage : Classifier le contenu d'un panier, d'une cagette
- Complexité : Dataset avec bounding boxes nécessaire, architecture plus complexe
- Bénéfice : Passage de classification simple à détection multi-objets

2. Mode offline avec TensorFlow Lite :

- Approche : Conversion du modèle en .tflite pour inférence on-device
- Avantages : Pas besoin d'internet, latence réduite, confidentialité améliorée
- Compromis : Précision potentiellement légèrement réduite
- Complexité technique : Quantization et optimisation nécessaires

3. Détection de fraîcheur et qualité :

- Objectif : Évaluer l'état (frais, mûr, trop mûr, pourri)
- Dataset nécessaire : Images annotées avec état de maturité
- Application : Réduction du gaspillage alimentaire, aide à l'achat
- Recherche : Identifier les features visuelles de la maturité

4. Informations nutritionnelles :

- Intégration : API nutritionnelle (USDA FoodData Central, Open Food Facts)
- Affichage : Calories, macronutriments, vitamines après classification
- Extension : Tracking calorique, suggestions de recettes
- Valeur ajoutée : Transformation en assistant nutritionnel

1.19.3 Long Terme (6-12 mois)

Des extensions ambitieuses transformant fondamentalement l'application :

1. Assistant culinaire intelligent :

- Vision : Application complète d'aide culinaire et nutritionnelle
- Fonctionnalités :
 - Reconnaissance d'ingrédients + suggestions de recettes
 - Génération de listes de courses optimisées
 - Tracking nutritionnel automatisé
 - Alertes de péremption basées sur détection de fraîcheur
- Partenariats : APIs de recettes, nutritionnistes

2. Version B2B professionnelle :

- Cibles : Supermarchés, restaurants, industries agroalimentaires
- Cas d'usage :
 - Inventaire automatisé de stocks
 - Contrôle qualité en chaîne de production
 - Optimisation de commandes basée sur détection
- Modèle économique : Licensing, SaaS

3. Reconnaissance d'aliments généralisée :

- Extension : Au-delà des fruits/légumes crus → plats cuisinés, produits transformés
- Dataset : Nécessite des millions d'images (Food-101, UPMC Food-101, etc.)
- Complexité : Variations énormes (même plat peut avoir 1000 apparences différentes)
- Ambition : Concurrence avec Google Lens, Yummly

4. Réalité augmentée :

- Vision : Classification en temps réel sur flux vidéo
- Affichage : Overlay AR avec informations superposées
- Technologies : ARCore/ARKit, TensorFlow Lite
- Expérience : Pointer la caméra → identification instantanée

1.20 Impact et Applications

1.20.1 Applications Concrètes

FruitVision AI, dans sa forme actuelle ou étendue, peut servir dans plusieurs contextes réels :

1. Éducation alimentaire :

- Cible : Enfants, écoles, programmes nutritionnels
- Usage : Apprendre à identifier les fruits et légumes de manière ludique
- Gamification : Challenges, badges, quiz interactifs

2. Nutrition et santé :

- Cible : Personnes suivant un régime, diabétiques, sportifs

- Usage : Tracking automatique de consommation de fruits/légumes
- Intégration : Apps de fitness (MyFitnessPal, Lose It !)

3. Commerce et distribution :

- Cible : Supermarchés, marchés, caisses automatiques
- Usage : Identification automatique pour pesée et facturation
- Bénéfice : Réduction du temps d'attente, automatisation

4. Agriculture et qualité :

- Cible : Producteurs, coopératives, centres de tri
- Usage : Contrôle qualité automatisé, tri par maturité
- Impact : Réduction du gaspillage, optimisation de la chaîne

1.20.2 Contribution au Domaine

Ce projet contribue au domaine de l'IA appliquée de plusieurs manières :

- **Démonstration pédagogique** : Preuve empirique accessible de l'efficacité du transfer learning
- **Template réutilisable** : Architecture complète que d'autres peuvent adapter pour leurs projets
- **Open-source potentiel** : Code bien structuré et documenté, prêt à être partagé
- **Vulgarisation** : Rendre l'IA concrète et compréhensible à travers une application tangible

1.21 Leçons Apprises

1.21.1 Techniques

1. Transfer learning > Tout le reste :

Pour des projets avec datasets limités (<10K images), le transfer learning n'est pas optionnel - c'est essentiel. L'amélioration de 48% à 93% dans notre cas est typique et justifie pleinement cette approche.

2. Architecture matters :

Choisir l'architecture appropriée au problème est fondamental. Les CNN pour la vision ne sont pas juste "meilleurs" - ils sont structurellement adaptés. Utiliser un ANN pour des images est une erreur conceptuelle, pas juste une question de performance.

3. Données > Modèle :

Un modèle simple avec beaucoup de données battra presque toujours un modèle complexe avec peu de données. L'augmentation aide, mais ne remplace pas un vrai dataset large et varié.

4. Preprocessing cohérent :

La cohérence entre entraînement et inférence est critique. La même normalisation et le même redimensionnement doivent être appliqués, sinon les performances s'effondrent.

5. Validation rigoureuse :

Séparer correctement train/validation, utiliser l'early stopping, monitorer les métriques de validation - ces pratiques évitent le surapprentissage et garantissent que les résultats sont fiables.

1.21.2 Développement

1. **Microservices = Flexibilité :**

La séparation frontend/backend facilite énormément le développement, les tests, et l'évolution. Chaque composant peut être développé, testé, et déployé indépendamment.

2. **Flutter pour prototypage rapide :**

Flutter permet de créer rapidement des prototypes fonctionnels et esthétiques multiplateformes. Le hot reload accélère l'itération UI de manière spectaculaire.

3. **BaaS simplifie la vie :**

Firebase (ou équivalents) pour l'authentification évite des semaines de développement et garantit la sécurité. Ne pas réinventer la roue pour les fonctionnalités standard.

4. **Documentation automatique :**

FastAPI générant automatiquement Swagger UI économise des heures de documentation manuelle et facilite les tests. Investir dans des outils qui documentent automatiquement.

5. **UX > Technologie :**

Un modèle à 95% de précision avec une UI confuse est moins utile qu'un modèle à 85% avec une excellente UX. L'expérience utilisateur doit être au centre des préoccupations.

1.21.3 Gestion de Projet

1. **MVP d'abord :**

Commencer par un Minimum Viable Product fonctionnel, puis itérer. Viser la perfection dès le départ mène à la paralysie.

2. **Tester tôt et souvent :**

Des tests réguliers à chaque étape permettent de détecter les problèmes tôt quand ils sont faciles à corriger. Attendre la fin pour tout tester est une erreur.

3. **Documenter en continu :**

Documenter au fur et à mesure, pas à la fin. Les décisions et raisonnements sont frais en mémoire et la documentation est beaucoup plus précise.

4. **Prioriser impitoyablement :**

Avec des délais académiques, il faut savoir quelles fonctionnalités sont essentielles et lesquelles peuvent attendre. Mieux vaut 5 fonctionnalités bien faites que 15 à moitié terminées.

5. **Apprendre des échecs :**

Le CNN custom avec 48% n'est pas un "échec" - c'est une validation que le transfer learning était la bonne approche. Les résultats négatifs enseignent autant que les positifs.

1.22 Mot de la Fin

Le développement de FruitVision AI a été une expérience extrêmement enrichissante, permettant d'appliquer concrètement les connaissances théoriques en intelligence artificielle, développement mobile, et architecture logicielle dans un projet cohérent et abouti.

Les résultats obtenus - notamment la précision remarquable de 93,62% du modèle avec transfer learning - démontrent que des applications d'IA performantes peuvent être développées même avec

des ressources académiques limitées, à condition d'utiliser les bonnes techniques et de structurer le projet intelligemment.

Au-delà des aspects purement techniques, ce projet a renforcé plusieurs principes fondamentaux :

- **Le pragmatisme prime sur la complexité** : Utiliser ce qui fonctionne (transfer learning, Firebase, Flutter) plutôt que chercher la solution la plus sophistiquée
- **L'itération mène à l'excellence** : Développer progressivement (CNN custom → transfer learning), tester fréquemment, améliorer continuellement
- **La documentation facilite tout** : Un projet bien documenté est plus facile à maintenir, à étendre, et à présenter
- **L'utilisateur est central** : La technologie la plus avancée sans bonne UX reste inutilisée - le design et l'expérience utilisateur sont aussi importants que l'algorithme

FruitVision AI constitue une base solide qui peut être étendue dans de nombreuses directions. Les perspectives d'amélioration identifiées - détection multi-objets, mode offline, détection de fraîcheur, assistant culinaire - montrent le potentiel d'évolution vers une application réellement utile au quotidien.

Ce projet illustre comment l'intelligence artificielle, lorsqu'elle est bien conçue et intégrée, peut rendre la technologie plus accessible et utile. La classification automatique d'images, qui semblait de la science-fiction il y a une décennie, est aujourd'hui à la portée de tout développeur disposant des bons outils et connaissances.

L'avenir de l'IA mobile est prometteur, et FruitVision AI en est une démonstration concrète. Le code développé, l'architecture conçue, et les leçons apprises peuvent servir de fondation pour de futurs projets encore plus ambitieux.

Enfin, ce projet a démontré qu'avec de la curiosité, de la persévérance, et une méthodologie rigoureuse, il est possible de créer des applications intelligentes qui allient performance technique et utilité pratique. C'est cette combinaison qui fera le succès des applications de demain.

Annexe A

Annexes Techniques

A.1 Installation et Configuration

A.1.1 Prérequis Système

- **Système d'exploitation** : Windows 10/11, macOS 10.15+, ou Linux (Ubuntu 20.04+)
- **RAM** : Minimum 8 GB (16 GB recommandé pour l'entraînement)
- **Espace disque** : Minimum 5 GB disponibles
- **Connexion internet** : Requisite pour télécharger les dépendances et utiliser l'application

A.1.2 Configuration Backend

Le backend Python nécessite Python 3.10+ et les dépendances listées dans `requirements.txt` :

- fastapi
- uvicorn
- python-multipart
- tensorflow (2.13+)
- pillow
- numpy

Les modèles pré-entraînés (`cnn_model.h5`, `ann_model.h5`) doivent être placés dans le dossier `models/`.

A.1.3 Configuration Frontend

Flutter 3.x doit être installé avec les packages définis dans `pubspec.yaml` :

- `firebase_core` : 3.0.1
- `firebase_auth` : 5.3.3
- `http` : 1.2.2
- `image_picker` : 0.8.6

Firebase doit être configuré avec un projet créé sur Firebase Console et les fichiers de configuration générés par FlutterFire CLI.

A.2 Structure du Code

A.2.1 Organisation Backend

Le code backend est centralisé dans `app.py` avec :

- Initialisation FastAPI et CORS
- Chargement des modèles au startup
- Endpoints de prédiction (`/predict`, `/predict/cnn`, `/predict/ann`)
- Pipeline de preprocessing
- Gestion des erreurs

A.2.2 Organisation Frontend

Le code Flutter est organisé par fonctionnalité :

- `main.dart` : Point d'entrée, routes
- `login_screen.dart`, `register_screen.dart` : Authentification
- `HomePage.dart` : Page d'accueil
- `CNNPage.dart`, `ANNPPage.dart` : Classification
- `menu.dart` : Navigation drawer
- `firebase_options.dart` : Configuration Firebase

Annexe B

Glossaire

Accuracy Proportion de prédictions correctes sur l'ensemble total de prédictions

AI (Artificial Intelligence) Intelligence Artificielle - Capacité d'une machine à imiter l'intelligence humaine

ANN (Artificial Neural Network) Réseau de Neurones Artificiel - Modèle fully connected inspiré du cerveau

API (Application Programming Interface) Interface permettant la communication entre applications

Augmentation de données Techniques pour augmenter artificiellement la taille d'un dataset en appliquant des transformations

Batch Size Nombre d'échantillons traités simultanément pendant l'entraînement

CNN (Convolutional Neural Network) Réseau de Neurones Convolutif - Architecture spécialisée pour les images

Dropout Technique de régularisation désactivant aléatoirement des neurones

Epoch Une passe complète sur l'ensemble des données d'entraînement

Feature Extraction Première phase du transfer learning où seule la nouvelle tête est entraînée

Fine-tuning Phase où les dernières couches du modèle pré-entraîné sont réentraînées

Firebase Plateforme BaaS de Google pour le développement mobile

Flutter Framework multiplateforme de Google pour applications mobiles

Loss Function Fonction mesurant l'écart entre prédictions et valeurs réelles

Overfitting Surapprentissage - Le modèle mémorise sans généraliser

Transfer Learning Réutilisation d'un modèle pré-entraîné pour une nouvelle tâche

Validation Set Ensemble de données pour évaluer le modèle pendant l'entraînement

Annexe C

Bibliographie

Bibliographie

- [1] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- [2] LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278-2324.
- [3] Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. *Advances in Neural Information Processing Systems*, 25.
- [4] Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L. C. (2018). MobileNetV2 : Inverted residuals and linear bottlenecks. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 4510-4520.
- [5] Yosinski, J., Clune, J., Bengio, Y., & Lipson, H. (2014). How transferable are features in deep neural networks? *Advances in Neural Information Processing Systems*, 27.
- [6] Pan, S. J., & Yang, Q. (2009). A survey on transfer learning. *IEEE Transactions on Knowledge and Data Engineering*, 22(10), 1345-1359.
- [7] Shorten, C., & Khoshgoftaar, T. M. (2019). A survey on image data augmentation for deep learning. *Journal of Big Data*, 6(1), 1-48.
- [8] Google. (2024). *Flutter Documentation*. Consulté sur <https://docs.flutter.dev>
- [9] Ramírez, S. (2024). *FastAPI Documentation*. Consulté sur <https://fastapi.tiangolo.com>
- [10] Google. (2024). *TensorFlow Documentation*. Consulté sur <https://www.tensorflow.org>
- [11] Google. (2024). *Firebase Documentation*. Consulté sur <https://firebase.google.com/docs>
- [12] Chollet, F. et al. (2024). *Keras Documentation*. Consulté sur <https://keras.io>